

Unit-1: Basic of Java

Main Topic	Subtopics	What You Must Understand
1. Java Platform and Architecture	JVM (Java Virtual Machine)	Role of JVM in executing Java bytecode
	JDK (Java Development Kit)	Tools required to develop Java applications
	JRE (Java Runtime Environment)	Runtime environment needed to run Java programs
	Java Platform concept	Write Once Run Anywhere (WORA)
	Java compilation process	.java → .class → JVM execution
	Java architecture	Interaction between JDK, JRE, JVM
2. Java Program Structure	Basic syntax of Java	Structure of class and main method
	Package declaration	Organizing Java classes
	Import statements	Using external libraries
	Class definition	Blueprint of objects
	Main method	Entry point of Java program
3. Compilation and Execution	Writing a Java program	Creating .java file
	Compilation	Using javac compiler
	Bytecode generation	.class file creation
	Program execution	Running using java command
4. Java Data Types	Primitive data types	byte, short, int, long
	Floating point types	float, double
	Character type	char
	Boolean type	boolean
	Non-primitive types	String
	Arrays	Collection of same type elements
5. Variables	Variable declaration	Defining variables
	Variable initialization	Assigning initial value
	Types of variables	Local, Instance, Static
	Scope of variables	Lifetime and accessibility
6. Java Keywords	Reserved keywords	class, public, static, void, etc.
	Usage rules	Keywords cannot be used as identifiers
7. Literals	Integer literals	Example: 10, 100
	Floating literals	Example: 3.14
	Character literals	'A'
	String literals	"Hello"

	Boolean literals	true, false
8. Operators	Arithmetic operators	+, -, *, /, %
	Relational operators	>, <, >=, <=, ==, !=
	Logical operators	&&, ^
	Assignment operators	=, +=, -=, *=
	Unary operators	++, --
	Bitwise operators	&, ^
9. Operator Precedence & Associativity	Order of operator evaluation	Which operator executes first
	Associativity rules	Left-to-right or right-to-left evaluation
10. Type Conversion	Implicit conversion (Widening)	Automatic type conversion
	Explicit conversion (Casting)	Manual type conversion
	Primitive type casting	Example: double → int

Introduction to Java

Java is a **high-level, object-oriented, platform-independent programming language** used to develop a wide range of applications such as web apps, mobile apps, enterprise systems, and embedded systems.

- Developed by **James Gosling** at **Sun Microsystems**
- Released in **1995**
- Follows the principle:
"Write Once, Run Anywhere (WORA)"

Key Idea: Java programs are compiled into **bytecode**, which runs on the **Java Virtual Machine (JVM)**, making Java platform-independent.

2. History of Java

Timeline

Year	Event
1991	Project "Oak" started by James Gosling
1995	Renamed to Java and officially released
1996	First version (JDK 1.0) launched
2009	Sun Microsystems acquired by Oracle Corporation
Present	Java widely used in enterprise, Android, cloud

Key Points

- Initially designed for **embedded systems (TV, devices)**
- Later adapted for **internet programming**
- Became popular due to **security and portability**

3. Features of Java

1. Simple

- Syntax similar to C/C++
- No pointers, no operator overloading
- Easy to learn

2. Object-Oriented

- Everything revolves around **objects**
- Supports:
 - Encapsulation
 - Inheritance

- Polymorphism
- Abstraction

3. Platform Independent

- Java code runs on any OS via JVM
- Bytecode ensures portability

4. Secure

- No direct memory access
- Bytecode verification
- Security manager

5. Robust

- Strong memory management
- Automatic garbage collection
- Exception handling

6. Multithreaded

- Supports concurrent execution
- Improves performance

7. High Performance

- Uses **Just-In-Time (JIT) compiler**
- Faster than interpreted languages

8. Distributed

- Supports network-based applications
- APIs for distributed computing

9. Dynamic

- Classes loaded at runtime
- Supports dynamic linking

4. Components of Java

1. JDK (Java Development Kit)

- Full package for development
- Includes:
 - Compiler (javac)
 - JRE
 - Tools

2. JRE (Java Runtime Environment)

- Provides environment to run Java programs
- Includes JVM + libraries

3. JVM (Java Virtual Machine)

- Executes bytecode
- Platform-dependent
- Core of Java portability

Diagram Understanding

Java Source Code (.java)



Compiler (javac)



Bytecode (.class)



JVM



Machine Code (OS specific)

5. Advantages of Java

Advantage	Explanation
Platform Independent	Run anywhere with JVM
Secure	No pointer manipulation
Robust	Error handling + GC

Multithreading	Efficient execution
Rich API	Large standard library
Scalable	Suitable for large systems
Community Support	Huge developer base

6. Characteristics of Java

These are theoretical properties often asked in exams.

1. Architecture Neutral

- Bytecode not dependent on CPU architecture

2. Portable

- Same program runs on different systems

3. Interpreted + Compiled

- Compiled to bytecode
- Interpreted by JVM

4. Distributed

- Built-in networking support

5. High Level Language

- Abstracts hardware details

6. Automatic Memory Management

- Garbage collector handles memory

Difference Between POP and OOP:

POP (Procedure Oriented Programming)	OOP (Object Oriented Programming)
In Procedure Oriented Programming, the main focus is on functions and procedures to perform operations on data.	In Object Oriented Programming, the main focus is on objects that combine both data and methods together.
In POP, the program is divided into small functions where each function performs a specific task.	In OOP, the program is divided into objects which represent real-world entities.
In POP, data is generally shared globally and can be accessed by any function in the program.	In OOP, data is hidden within objects and can only be accessed through methods, ensuring data security.
In POP, there is no concept of data hiding, so it is less secure.	In OOP, data hiding is achieved using encapsulation, making the program more secure.
In POP, the design approach follows a top-down approach where the problem is broken into smaller functions.	In OOP, the design approach follows a bottom-up approach where smaller objects are combined to form a complete system.
In POP, functions are independent and do not have direct relation with the data they operate on.	In OOP, methods are directly related to the data (objects) they operate on.
In POP, adding new data types or modifying existing code is difficult and may affect the entire program.	In OOP, new objects and classes can be added easily without affecting existing code.
In POP, reusability of code is limited because functions are not designed for reuse across programs.	In OOP, code reusability is high due to concepts like inheritance and polymorphism.
In POP, it is not suitable for large and complex applications due to lack of modular structure.	In OOP, it is suitable for large and complex applications due to its modular and structured design.
In POP, real-world entities are not represented directly in the program.	In OOP, real-world entities are represented as objects, making the design more realistic.
In POP, there is no support for inheritance and polymorphism.	In OOP, inheritance and polymorphism are key features that enhance flexibility and reusability.
In POP, debugging and maintenance become difficult as the program grows larger.	In OOP, debugging and maintenance are easier because of modular structure and clear separation of components.

Java Platform and Architecture

This part explains **how Java programs actually run inside a computer.**

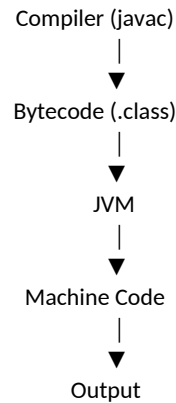
It is one of the **most important theory questions in mid-sem.**

1. Overview of Java Platform Architecture

Before going deep, understand the **complete execution flow.**

Java Program (.java)





Example:

```

class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
  
```

Compilation

javac Hello.java

Output file

Hello.class

Execution

java Hello

The .class file runs inside **JVM**, not directly on OS.

2. JVM (Java Virtual Machine)

Definition : JVM is a virtual machine that executes Java bytecode.

It acts like an **intermediate layer between Java program and operating system.**

Why JVM Exists: Different computers have different architectures:

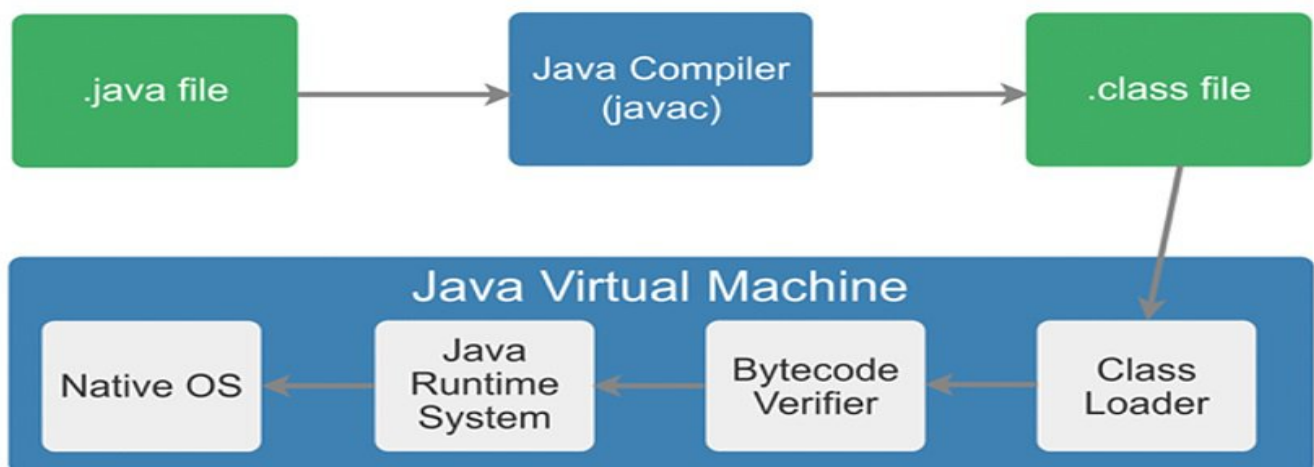
OS	CPU Architecture
Windows	x86
Linux	ARM
Mac	Apple Silicon

Java solves this problem by compiling code into **bytecode**, which is platform-independent.

JVM converts bytecode into **machine code for that system.**

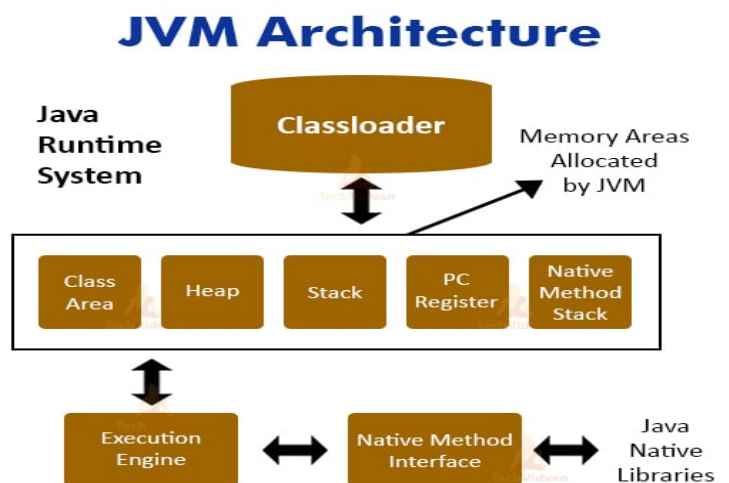
This is why Java supports: Write Once Run Anywhere

What is Java bytecode?



JVM Architecture

Component	Purpose
Class Loader	Loads .class files
Method Area	Stores class metadata
Heap	Stores objects
Stack	Stores method calls
PC Register	Tracks instruction execution
Native Method Stack	Runs native methods
Execution Engine	Executes bytecode



1. Class Loader

Purpose: Load .class files into JVM memory.

Steps:

Step	Meaning
Loading	Load class file
Linking	Verify bytecode
Initialization	Execute static blocks

Example:

When program runs: java Hello

Class loader loads: Hello.class

2. Runtime Data Areas

JVM memory is divided into multiple areas:

1. Heap

Used for:

- Objects
- Arrays

Example : Student s = new Student();

Object stored in **heap memory**.

2. Stack

Stores:

- Method calls
- Local variables

Example : int a = 10;

Stored in stack.

3. Method Area

Stores:

- Class structure
- Static variables
- Method code

Program Counter (PC Register)

Tracks: Current instruction executed

Each thread has its own PC register.

Native Method Stack

Used for **native methods written in C/C++**.

Example : System.out.println()

Internally calls native code.

3. Execution Engine

Responsible for executing bytecode.

It contains:

Component	Role
Interpreter	Executes code line by line
JIT Compiler	Converts bytecode into machine code

JIT improves performance

Example of Execution

Program: `System.out.println("Hello");`

Steps:

1. Class loader loads class
2. Bytecode verified
3. JVM stores data in memory
4. Execution engine runs code
5. Output displayed

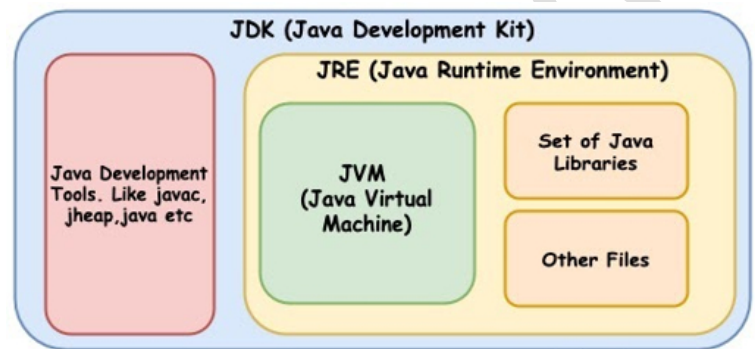
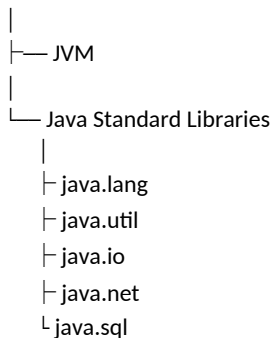
3. JRE (Java Runtime Environment)

Definition : JRE provides runtime environment to execute Java programs.

It contains: JVM + Java Libraries

JRE Structure

JRE



techcrashcourse.com

Purpose of JRE

JRE allows **running Java applications** but not developing them.

Example: A user running a Java application only needs **JRE installed**.

Example: Minecraft game

Runs on JRE.

Components of JRE

Component	Purpose
JVM	Executes bytecode
Class Libraries	Predefined Java functions
Runtime Files	Required execution files

Example

Code: `System.out.println("Hello");`

`System.out.println()` comes from:

`java.lang` library

This library is part of **JRE**.

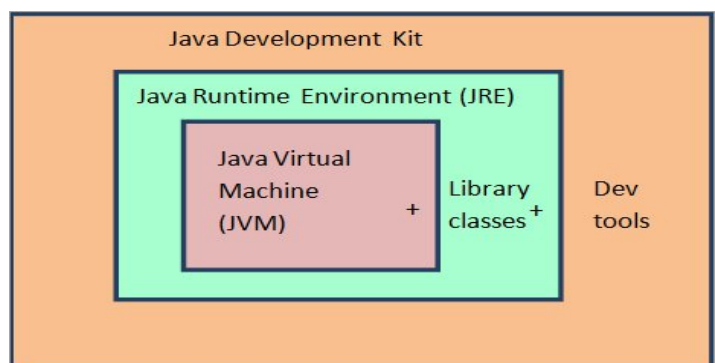
4. JDK (Java Development Kit)

Definition : JDK is a **complete toolkit** used to develop Java applications.

It contains: JDK = JRE + Development Tools

JDK Architecture

JDK



Important JDK Tools

Tool	Purpose
javac	Compiles Java code
java	Runs program
javadoc	Generates documentation
jar	Creates Java archive
jdb	Debugger
javap	Bytecode viewer

5. Relationship Between JVM, JRE, and JDK

Best way to remember:

JDK

└─ JRE

└─ JVM

Comparison Table

Feature	JVM	JRE	JDK
Full Form	Java Virtual Machine	Java Runtime Environment	Java Development Kit
Purpose	Executes bytecode	Provides runtime environment	Develop Java programs
Contains	Execution engine	JVM + libraries	JRE + development tools
Used by	End user / runtime	Application users	Developers

6. Real World Analogy : Think of Java like car manufacturing.

Concept	Example
Java Program	Car design
JDK	Car factory
JRE	Car engine system
JVM	Engine inside car

1. Java Platform Concept

Definition: A **platform** is the environment where programs run.

It normally includes:

Component	Meaning
Operating System	Windows, Linux, macOS
Hardware	CPU, memory
Runtime environment	Software needed to execute programs

What Makes Java a Platform

Java provides its **own runtime environment**. Unlike languages like C or C++, Java programs do not run directly on the operating system.

Instead they run on: Java Virtual Machine (JVM)

Therefore Java itself acts as a **software platform**.

Why Java is Platform Independent

Most programming languages compile code directly into **machine code**.

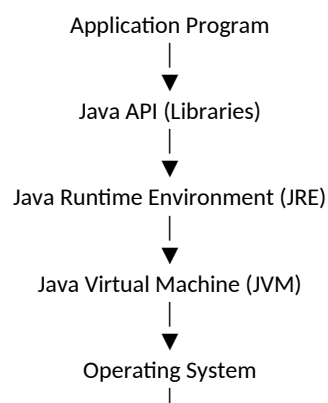
Example: C program → machine code → runs only on same OS

Java works differently : Java code → Bytecode → JVM → Machine code

Because every operating system has its own JVM, the same program can run everywhere.

This concept is called: Write Once Run Anywhere (WORA)

Java Platform Layers



▼
Hardware

Example : Java program written on Windows can run on:

System	Reason
Windows	JVM for Windows
Linux	JVM for Linux
macOS	JVM for macOS

Only JVM changes, program remains same.

Role of JIT Compiler

Inside JVM there is a **Just-In-Time (JIT) compiler**.

Purpose:

Component	Function
Interpreter	Executes code line by line
JIT Compiler	Converts bytecode to machine code

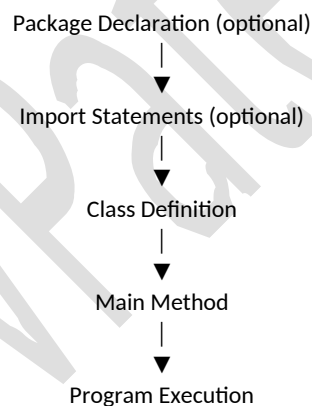
JIT improves performance by compiling frequently used code.

1. Overall Structure of a Java Program

A Java program is organized in the following order:

```
package packageName;
import packageName.ClassName;
class ClassName {
    variables;
    methods;
    public static void main(String[] args) {
        // program execution starts here
    }
}
```

Complete Structure Flow



Java Naming Conventions (Standard Rules for Exam)

Java follows **strict naming conventions** to keep code readable, consistent, and professional. These are very important for exams and practical coding.

1. Class Name Standard

Rules

- Class name should start with a **capital letter**
- Use **PascalCase** (first letter of every word capital)
- Should be a **noun (object name)**

Examples

Correct	Incorrect
Student	student
BankAccount	bank_account
EmployeeDetails	employeedetails

Explanation: Class names represent **real-world entities**, so they are written like proper nouns.

2. Package Name Standard

Rules

- Package name should be **lowercase only**

- Use **reverse domain name** (recommended)
- Words separated by dots (.)

Examples

Correct	Incorrect
com.company.project	Company.Project
org.example.util	org.Example.Util

Explanation : Packages are used to **organize classes**, so lowercase avoids conflicts.

3. Variable Name Standard

Rules

- Use **camelCase**
- Start with a **lowercase letter**
- Should be **meaningful**
- Avoid single-letter names (except loops)

Examples

Correct	Incorrect
studentName	StudentName
totalMarks	total_marks
accountBalance	acc

Explanation: Variables represent **data**, so names should clearly describe their purpose.

4. Method Name Standard

Rules

- Use **camelCase**
- Start with a **lowercase letter**
- Should be a **verb (action)**

Examples

Correct	Incorrect
calculateTotal()	CalculateTotal()
getName()	get_name()
displayResult()	resultDisplay()

Explanation: Methods represent **actions**, so they are written as verbs.

5. Constant Name Standard

Rules

- Use **UPPERCASE letters**
- Words separated by **underscore (_)**

Examples

Correct	Incorrect
MAX_VALUE	maxValue
PI_VALUE	piValue

6. Interface Name Standard

Rules

- Same as class name (**PascalCase**)
- Often adjective or capability-based

Examples

Correct	Incorrect
Runnable	runnable
Serializable	serializable

7. Summary Table (Quick Recall)

Element	Naming Style	Example
Class	PascalCase	StudentData
Package	lowercase	com.example.app
Variable	camelCase	totalMarks
Method	camelCase (verb)	calculateSum()
Constant	UPPER_CASE	MAX_SIZE
Interface	PascalCase	Runnable

2. Basic Syntax of Java

Java syntax refers to **rules for writing Java programs**.

Important Syntax Rules

Rule	Explanation
------	-------------

Case sensitive	Main and main are different
Class name must match file name	Hello.java → class Hello
Every statement ends with ;	Example: int a = 10;
Code written inside classes	Java is pure class-based
Execution starts from main()	Entry point of program

3. Package Declaration

Definition : A **package** is a collection of related classes.

It is used to **organize large Java programs**.

Example packages in Java:

- java.lang
- java.util
- java.io
- java.net

Syntax : package packageName;

Example : package com.company.project;

Example Program Using Package

```
package myprogram;
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello from package");
    }
}
```

Advantages of Packages

Advantage	Explanation
Organization	Group related classes
Avoid name conflict	Two classes can have same name in different packages
Access control	Control class visibility
Code reuse	Easy reuse of libraries

4. Import Statements

Definition : The **import statement** allows a program to use classes from other packages.

Without import, the full package name must be written.

Syntax

import packageName.ClassName;

Example : import java.util.Scanner;

Import Entire Package

import java.util.*;

Example Program

```
import java.util.Scanner;
class InputExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number:");
        int num = sc.nextInt();
        System.out.println(num);
    }
}
```

Types of Import

Type	Example
Single class import	import java.util.Scanner;
Entire package import	import java.util.*;

□ Ways to Take User Input in Java

Java provides multiple ways to take input. You should know **at least 3 methods**.

1. Using Scanner Class (Most Important)

Definition: Scanner is used to take input from keyboard (System.in).

Syntax

```
Scanner sc = new Scanner(System.in);
```

Example

```
import java.util.Scanner;
class InputExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number:");
        int num = sc.nextInt();

        System.out.println("Number: " + num);
        sc.close();
    }
}
```

Important Methods

Method	Use
nextInt()	Integer input
nextFloat()	Float input
nextDouble()	Double input
next()	Single word
nextLine()	Full sentence

2. Using BufferedReader Class (Faster, Advanced)

Definition : Used for faster input (mainly in competitive programming)

Syntax

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

Example

```
import java.io.*;
class InputExample {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter number:");
        int num = Integer.parseInt(br.readLine()); // type casting
        System.out.println("Number: " + num);
    }
}
```

Key Points

- Reads input as **String**
- Requires conversion (Integer.parseInt())

3. Using Command Line Arguments

Definition : Input is passed while running program

Example

```
class Main {
    public static void main(String[] args) {
        int num = Integer.parseInt(args[0]); // Type Casting
        System.out.println("Number: " + num);
    }
}
```

Run Command : java Main 10

Output : Number: 10

4. Using Console Class :

Example

```
import java.io.Console;
class Test {
    public static void main(String[] args) {
        Console c = System.console();
        String name = c.readLine("Enter name: ");
        System.out.println(name);
    }
}
```

Note

- Not supported in many IDEs
- Works in terminal

5. Class Definition

Definition: A **class** is a blueprint for creating objects.

It defines:

- variables (data members)
- methods (functions)

Syntax

```
class ClassName {
    data members;
    methods;
}
```

Class Components

Component	Description
Class name	Name of blueprint
Variables	Store data
Methods	Perform actions
Constructors	Initialize objects

6. Main Method

Definition : The **main() method** is the entry point of a Java program.

When the program runs, JVM starts execution from main().

Syntax

```
public static void main(String[] args)
```

Explanation of Each Keyword

Keyword	Meaning
public	Accessible everywhere
static	Method belongs to class
void	Does not return value
main	Method name
String[] args	Command line arguments

7. Example of Complete Java Program

```
package myapp;
import java.util.Scanner;

class Calculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        System.out.println("Sum = " + (a + b));
    }
}
```

Java Data Types

Definition: A **data type** specifies the type of data a variable can store.

Example: `int age = 20;`

Here:

- `int` → data type
- `age` → variable
- `20` → value

Classification of Java Data Types

Java data types are divided into two categories.

Category	Types
Primitive Data Types	byte, short, int, long, float, double, char, boolean
Non-Primitive Data Types	String, Arrays, Classes, Objects

4.1 Primitive Data Types

Primitive types store **simple values directly in memory**.

They are predefined by Java.

Integer Types

Used to store **whole numbers**.

Type	Size	Range	Example
byte	1 byte	-128 to 127	byte a = 10;
short	2 bytes	-32,768 to 32,767	short b = 1000;
int	4 bytes	-2^{31} to $2^{31}-1$	int num = 100;
long	8 bytes	very large range	long population = 9000000000L;

Example

```
class IntegerTypes {  
    public static void main(String[] args) {  
        byte a = 10;  
        short b = 200;  
        int c = 5000;  
        long d = 100000L;  
        System.out.println(a + " " + b + " " + c + " " + d);  
    }  
}
```

Floating Point Types

Used to store **decimal numbers**.

Type	Size	Precision
float	4 bytes	~7 decimal digits
double	8 bytes	~15 decimal digits

Example:

```
float price = 10.5f;
```

```
double pi = 3.14159;
```

Program example:

```
class FloatExample {  
    public static void main(String[] args) {  
        float f = 12.5f;  
        double d = 10.45678;  
        System.out.println(f);  
        System.out.println(d);  
    }  
}
```

Character Type

Stores a **single character**.

Type	Size	Example
char	2 bytes	'A', 'b', '1'

Java uses **Unicode** for characters.

Example:

```
char grade = 'A';
```

Program example:

```
class CharExample {  
    public static void main(String[] args) {  
        char letter = 'D';  
        System.out.println(letter);  
    }  
}
```

Boolean Type

Stores **true or false values**.

Type	Values
boolean	true or false

Example:

```
boolean isJavaEasy = true;
```

Program example:

```
class BooleanExample {  
    public static void main(String[] args) {  
        boolean result = true;  
        System.out.println(result);  
    }  
}
```

```
}  
}
```

4.2 Non-Primitive Data Types

Non-primitive types store **references to objects** rather than the actual value.

Examples include:

Type	Description
String	Stores text
Arrays	Stores multiple values
Classes	Blueprint of objects
Objects	Instance of class

String

String is used to store **text or sequence of characters**.

Example:

```
String name = "Dhruv";
```

Program example:

```
class StringExample {  
    public static void main(String[] args) {  
        String message = "Welcome to Java";  
        System.out.println(message);  
    }  
}
```

Arrays: An array stores **multiple values of the same type**.

Example: `int numbers[] = {10,20,30,40};`

Program example:

```
class ArrayExample {  
    public static void main(String[] args) {  
        int arr[] = {10,20,30};  
        for(int i=0; i < arr.length; i++){  
            System.out.println(arr[i]);  
        }  
    }  
}
```

Output: 10, 20, 30

Primitive vs Non-Primitive Types

Feature	Primitive	Non-Primitive
Stores	Actual value	Reference
Size	Fixed	Depends on object
Examples	int, float	String, Array

Variables

Definition: A **variable** is a **named memory location** used to store data.

Example: `int age = 20;`

Here:

- `age` → variable name
- `20` → stored value

Variable Declaration

Variable declaration means **creating a variable with a data type**.

Syntax: `datatype variableName;`

Example:

```
int number;
```

```
double salary;
```

```
char grade;
```

Variable Initialization

Assigning a value to a variable.

Example: `int number = 10;`

Program example:

```
class VariableExample {  
    public static void main(String[] args) {  
        int a;
```

```

    a = 10;
    System.out.println(a);
}
}

```

Types of Variables

Java has three main types of variables.

Type	Location	Example
Local Variable	Inside method	int x;
Instance Variable	Inside class	int age;
Static Variable	Shared by all objects	static int count;

1. Local Variables

Declared inside methods.

Example:

```

class LocalVar {
    public static void main(String[] args) {
        int x = 10;
        System.out.println(x);
    }
}

```

Characteristics:

- Created when method starts
- Destroyed when method ends

2. Instance Variables

Declared inside class but outside methods.

Each object has its **own copy**.

Example:

```

class Student {
    int roll;
    String name;
}

```

Example usage:

```

Student s1 = new Student();
s1.roll = 101;

```

3. Static Variables

Shared by **all objects of the class**.

Example:

```

class Counter {
    static int count = 0;
}

```

Every object uses the **same variable**.

Scope of Variables

Scope refers to **where the variable can be accessed**.

Variable	Scope
Local	Inside method
Instance	Inside class
Static	Entire class

Lifetime of Variables

Variable	Lifetime
Local	Until method execution ends
Instance	Until object exists
Static	Until program ends

Example Program Demonstrating Variables

```

class Student {
    int roll;           // instance variable
    static String college = "GTU"; // static variable
    void display() {
        int marks = 90; // local variable
    }
}

```



```

        System.out.println(roll + " " + college + " " + marks);
    }
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 101;
        s1.display();
    }
}

```

Java Keywords

Definition : Keywords are reserved words in Java that have predefined meaning and cannot be used as identifiers (variable names, class names, etc.).

Example: `int number = 10;`

Here:

- `int` → keyword
- `number` → identifier

If we try:

`int class = 5;`

This causes an error because **class** is a keyword.

Characteristics of Java Keywords

Property	Explanation
Reserved words	Have fixed meaning
Cannot be used as variable names	Example: <code>int if = 10</code> is invalid
Case-sensitive	<code>Class</code> ≠ <code>class</code>
Used by compiler	Helps compiler understand program structure

Common Java Keywords

Java has **50+** keywords. Important ones for beginners:

Category	Keywords
Class related	<code>class</code> , <code>extends</code> , <code>implements</code>
Access control	<code>public</code> , <code>private</code> , <code>protected</code>
Data types	<code>int</code> , <code>char</code> , <code>boolean</code> , <code>double</code>
Control flow	<code>if</code> , <code>else</code> , <code>switch</code> , <code>case</code>
Looping	<code>for</code> , <code>while</code> , <code>do</code>
Exception handling	<code>try</code> , <code>catch</code> , <code>finally</code> , <code>throw</code>
Method control	<code>return</code> , <code>static</code> , <code>void</code>
Object creation	<code>new</code>

Example Program Using Keywords

```

class KeywordExample {
    public static void main(String[] args) {
        int number = 10;
        if(number > 5) {
            System.out.println("Number is greater than 5");
        }
    }
}

```

Keywords used here: `class`, `public`, `static`, `void`, `int`, `if`

Literals

Definition : A literal is a constant value assigned to a variable in a program.

Example: `int x = 10;`

Here: `10` → literal

Types of Literals

Literal Type	Example
Integer literal	<code>10</code> , <code>100</code> , <code>-50</code>
Floating literal	<code>3.14</code> , <code>10.5</code>
Character literal	<code>'A'</code> , <code>'b'</code>
String literal	<code>"Hello"</code>
Boolean literal	<code>true</code> , <code>false</code>

Integer Literals : Used to represent **whole numbers**.

Example: int a = 100;

Different formats:

Format	Example
Decimal	100
Binary	0b1010
Octal	012
Hexadecimal	0x1A

Example: int binary = 0b1010;

Value stored = **10**

Floating Point Literals: Represent **decimal numbers**.

Example:

float price = 10.5f;

double pi = 3.14159;

Important rule: float must end with 'f'

Example: float value = 5.5f;

Character Literals: Represent **single characters**.

Example: char grade = 'A';

Unicode example: char letter = '\u0041';

Output: A

String Literals : A **sequence of characters enclosed in double quotes**.

Example: String name = "Java Programming";

Boolean Literals

Represent logical values.

Example: boolean isJavaFun = true;

Example Program Using Literals

```
class LiteralExample {  
    public static void main(String[] args) {  
        int number = 100;  
        float price = 12.5f;  
        char grade = 'A';  
        String message = "Java";  
        boolean status = true;  
  
        System.out.println(number);  
        System.out.println(price);  
        System.out.println(grade);  
        System.out.println(message);  
        System.out.println(status);  
    }  
}
```

Operators

Definition : Operators are **symbols used to perform operations on variables and values**.

Example: int c = a + b;

Types of Operators in Java

Operator Type	Example
Arithmetic	+, -, *, /, %
Relational	>, <, >=, <=, ==, !=
Logical	!, &&,
Assignment	=, +=, -=, *=, /=
Unary	++, --
Bitwise	~, &,
Ternary	? :

Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning	Example
+	Addition	a + b

-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b

Example:

```
class ArithmeticExample {
    public static void main(String[] args) {
        int a = 10;
        int b = 3;

        System.out.println(a + b);
        System.out.println(a - b);
        System.out.println(a * b);
        System.out.println(a / b);
        System.out.println(a % b);
    }
}
```

Relational Operators

Used to compare values.

Operator	Meaning
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal
==	Equal
!=	Not equal

Example:

```
int a = 10;
int b = 5
System.out.println(a > b);
Output: true
```

Logical Operators

Used with **boolean expressions**.

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

Example:

```
boolean a = true;
boolean b = false;
System.out.println(a && b);
Output: false
```

Assignment Operators

Used to assign values.

Operator	Example
=	a = 5
+=	a += 5
-=	a -= 3
*=	a *= 2
/=	a /= 4

Example:

```
int a = 10;
a += 5;
Value becomes: 15
```

Unary Operators

Operate on **one operand**.

Operator	Meaning
++	Increment
--	Decrement

Example:

```
int x = 5; x++;
Now: x = 6
```

Bitwise Operators

Operate on **binary numbers**.

Operator	Meaning
&	AND
	OR
^	XOR
~	Complement

Example:

```
int a = 5;
```

```
int b = 3;
```

```
System.out.println(a & b);
```

Ternary Operator : Short form of **if-else**.

Syntax: condition ? value1 : value2

Example:

```
int a = 10;
```

```
int b = 20;
```

```
int max = (a > b) ? a : b;
```

Example Program Demonstrating Operators

```
class OperatorExample {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 5;  
        System.out.println("Addition: " + (a + b));  
        System.out.println("Greater: " + (a > b));  
        System.out.println("Logical AND: " + (a > 5 && b < 10));  
    }  
}
```

Operator Precedence

Definition: Operator precedence determines the order in which operators are evaluated in an expression.

When multiple operators appear in one expression, Java follows a **priority hierarchy**.

Example int result = 10 + 5 * 2;

Evaluation:

5 * 2 = 10

10 + 10 = 20

Output:

20

Reason: * has higher precedence than +

Java Operator Precedence Table

Priority	Operators	Description
Highest	() [] .	Parentheses, array access
2	++, --, !	Unary operators
3	*, /, %	Multiplication, division, modulus
4	+, -	Addition, subtraction
5	< > <= >=	Relational
6	== !=	Equality
7	&&	Logical AND
8	~	Logical not
9	?:	Ternary operator
Lowest	=, +=, -=	Assignment

Example Program

```
class PrecedenceExample {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 5;  
        int c = 2;  
        int result = a + b * c;  
  
        System.out.println(result);  
    }  
}
```

```
}  
}  
Output:  
20
```

Associativity

Definition: Associativity determines the direction in which operators are evaluated when they have the same precedence.

Types of Associativity

Type	Direction
Left to Right	Most operators
Right to Left	Assignment and unary operators

Left-to-Right Associativity Example

```
int result = 10 - 5 - 2;  
Evaluation:  
10 - 5 = 5  
5 - 2 = 3  
Output: 3
```

Right-to-Left Associativity Example

```
int a, b, c;  
a = b = c = 10;  
Evaluation:  
c = 10  
b = 10  
a = 10  
All variables become:  
10
```

Combined Example

```
class AssociativityExample {  
    public static void main(String[] args) {  
        int a = 20;  
        int b = 5;  
        int c = 2;  
        int result = a / b * c;  
  
        System.out.println(result);  
    }  
}
```

Evaluation:
 $20 / 5 = 4$
 $4 * 2 = 8$
Output: 8

Quick Difference

Concept	Meaning
Precedence	Which operator runs first
Associativity	Direction of evaluation

Type Conversion

Definition : Type conversion is the process of converting one data type into another.

Example:
 $\text{int} \rightarrow \text{double}$
 $\text{double} \rightarrow \text{int}$

Types of Type Conversion

Type	Description
Implicit Conversion	Automatic conversion
Explicit Conversion	Manual conversion

3.1 Implicit Type Conversion (Widening):

Automatic conversion performed by Java when **smaller data type converts into larger data type**.

Example: byte → short → int → long → float → double

Example

```
class WideningExample {  
    public static void main(String[] args) {  
        int num = 10;  
        double value = num;
```

```
        System.out.println(value);  
    }  
}
```

Output: 10.0

Widening Conversion Diagram

Byte → short → int → long → float → double

Integer

float

No data loss occurs.

3.2 Explicit Type Conversion (Casting)

When **larger data type is converted to smaller type**, Java requires explicit casting.

Example: double → int

Example

```
class NarrowingExample {  
    public static void main(String[] args) {  
        double value = 9.78;  
        int num = (int) value;  
        System.out.println(num);  
    }  
}
```

Output : 9

Decimal part is removed.

Primitive Type Casting

Syntax: datatype variable = (datatype) value or variable;

Example:

```
int num = (int) 5.9;
```

Result: 5

Example Program Demonstrating Type Conversion

```
class TypeConversionExample {  
    public static void main(String[] args) {  
        int a = 10;  
        double b = a;    // widening  
        double x = 9.5;  
        int y = (int) x;  // narrowing  
  
        System.out.println(b);  
        System.out.println(y);  
    }  
}
```

Output:

10.0

9

Key Differences

Feature	Widening	Narrowing
Conversion	Small → Large	Large → Small
Casting required	No	Yes
Data loss	No	Possible

Wrapper Classes in Java

Definition: Wrapper classes are **object representations** **Classes of primitive data types**.

They allow primitives to be used where **objects are required** (e.g., collections, OOP concepts).

Primitive → Wrapper Mapping

Primitive Type	Wrapper Class
int	Integer
float	Float
double	Double
char	Character
boolean	Boolean
long	Long
short	Short
byte	Byte

Why Wrapper Classes are Used

- Java collections (like ArrayList) store **objects only**
- Needed for **autoboxing / unboxing**
- Provide **utility methods** for conversion and operations
- Support **null values** (primitive cannot)

Example

```
class WrapperExample {  
    public static void main(String[] args) {  
        int a = 10;  
        // Boxing  
        Integer obj = Integer.valueOf(a);  
  
        // Autoboxing  
        Integer obj2 = a;  
  
        System.out.println(obj);  
        System.out.println(obj2);  
    }  
}
```

Important Methods of Wrapper Classes

1. valueOf()

Definition : Converts primitive or String into wrapper object.

Example

```
Integer obj = Integer.valueOf(10);  
Integer obj2 = Integer.valueOf("20");
```

2. parseXxx()

Definition : Converts String → primitive data type

Examples

```
int num = Integer.parseInt("100");  
double d = Double.parseDouble("10.5");
```

3. xxxValue()

Definition : Converts wrapper object → primitive data type

Examples

```
Integer obj = 50;  
int num = obj.intValue();
```

4. toString()

Definition: Converts value into String

Example

```
Integer obj = 100;  
String str = obj.toString();
```

5. compareTo()

Definition : Compares two wrapper objects

Example

```
Integer a = 10;  
Integer b = 20;  
System.out.println(a.compareTo(b)); // -1
```

Output Meaning

Result	Meaning
0	Equal
Negative	Less than
Positive	Greater than

6. equals()

Definition : Checks equality of two objects

Example

```
Integer a = 10;
Integer b = 10;
System.out.println(a.equals(b)); // true
```

7. MAX_VALUE & MIN_VALUE

Definition : Gives maximum and minimum values of data type

Example

```
System.out.println(Integer.MAX_VALUE);
System.out.println(Integer.MIN_VALUE);
```

8. Character Class Methods

Method	Use
isDigit()	Checks digit
isLetter()	Checks letter
isUpperCase()	Uppercase check
isLowerCase()	Lowercase check

Example

```
char ch = 'A';
System.out.println(Character.isUpperCase(ch));
```

9. Boolean Methods

```
Boolean b = Boolean.valueOf("true");
System.out.println(b.booleanValue());
```

Summary Table :

Method	Purpose
valueOf()	Primitive/String → Object
parseXxx()	String → Primitive
xxxValue()	Object → Primitive
toString()	Convert to String
compareTo()	Compare values
equals()	Check equality
MAX_VALUE	Maximum value
MIN_VALUE	Minimum value

Arrays in Java

◆ 1. What is an Array in Java?

An **array** is a container that stores **multiple values of the same type** in a single variable.

☞ Instead of writing:

```
int a = 10;
int b = 20;
int c = 30;
```

☞ You use:

```
int[] numbers = {10, 20, 30};
```

◆ 2. Why Arrays?

- Store multiple values efficiently
- Easy access using index
- Used in sorting, searching, algorithms
- Memory is **contiguous (continuous)**

◆ 3. Array Syntax

☞ **Declaration:** int[] arr;

☞ **Initialization:** arr = new int[5];

🔗 Combined: `int[] arr = new int[5];`

◆ 4. Indexing (Very Important)

- Array index starts from 0
- Ends at **size - 1**

Example:

```
int[] arr = {10, 20, 30, 40};
System.out.println(arr[0]); // 10
System.out.println(arr[2]); // 30
```

◆ 5. Memory Representation

Think like this:

Index	Value
0	10
1	20
2	30

🔗 Stored in continuous memory locations.

◆ 6. Types of Arrays

📦 Single-Dimensional Array

```
int[] arr = {1, 2, 3, 4};
```

📦 Multi-Dimensional Array (2D)

```
int[][] matrix = {
    {1, 2},
    {3, 4}
};
```

Access: `System.out.println(matrix[0][1]); // 2`

◆ 7. Input & Output Example

```
import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[3];
        // Input
        for(int i = 0; i < 3; i++) {
            arr[i] = sc.nextInt();
        }
        // Output
        for(int i = 0; i < 3; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

◆ 8. Looping Through Arrays

🔗 Using for loop

```
for(int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

🔗 Using enhanced for loop (for-each)

```
for(int num : arr) {
    System.out.println(num);
}
```

◆ 9. Important Property

`arr.length`

🔗 Gives total number of elements.

◆ 10. Default Values in Array

Data Type	Default Value
int	0
float	0.0

boolean	false
String	null

◆ 11. Common Operations

◆ Traversing

```
for(int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

◆ Sum of Array

```
int sum = 0;
for(int i = 0; i < arr.length; i++) {
    sum += arr[i];
}
System.out.println(sum);
```

◆ Find Maximum

```
int max = arr[0];
for(int i = 1; i < arr.length; i++) {
    if(arr[i] > max) {
        max = arr[i];
    }
}
System.out.println(max);
```

◆ 12. 2D Array Example

```
class Main {
    public static void main(String[] args) {
        int[][] arr = {
            {1, 2, 3},
            {4, 5, 6}
        };
        for(int i = 0; i < 2; i++) {
            for(int j = 0; j < 3; j++) {
                System.out.print(arr[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

◆ 13. Advantages

- ✓ Easy data management
- ✓ Fast access (O(1))
- ✓ Useful in algorithms
- ✓ Memory efficient

◆ 14. Disadvantages

- ✗ Fixed size or static size(java also support dynamic Arrays)
- ✗ Cannot store different data types
- ✗ Insertion/deletion is costly

◆ 15. Important Points for Exams ✗

- Arrays are **objects in Java**
- Stored in **heap memory**
- Index starts from **0**
- Size is **fixed**
- Use .length not length()